Tutorial 6

Core exercises

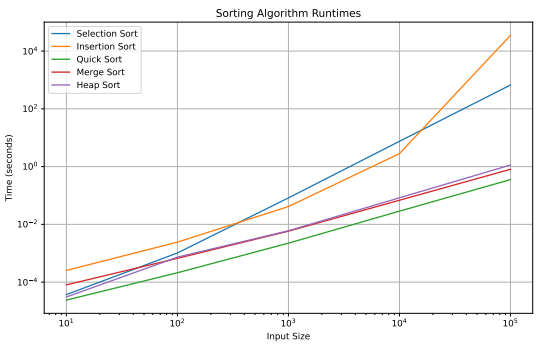
1. **Sorting Algorithms in Action**
   Sorting an array of data is a very common operation. In this exercise we will implement some of the most common algorithms generally used for this task.
   (a) Write a function implementing the selection sort algorithm. What is the disadvantage of selection sort?
   (b) Selection sort is not a very efficient method to sort (large) data sets. Implement <u>one</u> of the following sorting algorithms:
      1. Quicksort
      2. Merge sort
      3. Heapsort (suggestion: implement a function that constructs a heap of your data vector; then a second function with the actual sorting algorithm, which calls the first function)
   (c) Test your algorithms using a list of random numbers generated with any of the methods you developed for Tutorial 5. Plot the runtime of each algorithm versus the size of the array N for different N, starting at $N = 5$ and increasing exponentially to $N \geq 10^6$ (e.g., 6 distinct values for N, equally spaced in log space). Which algorithm works better in which range? Could you run both algorithm in a reasonable amount of time for every N?
   (d) Sort one of the lists of your random numbers and find the 16th, 50th (i.e. the median), and 84th percentiles. Now shuffle the array to a random order.

Solution:

(a) See slide 6 to how to implement selection sort. The disadvantage of selection sort is that it is slow, time $\propto N^2$.

(b) Again how to implement the different methods can be found on slide 9 (quicksort), slide 12 (merge sort) and slide 14 (heap sort)

(c) The result of the comparison is something like:



(d) This part consists of two parts, first order the list, and than find the 18 percent value to fidn the 18th percentile value. Now shuffle the array. This can be done using for example Fisher-Yates shuffle (see wikipedia), this method you take a random number between 1 and the length of your array N and determine which number goes to your random shuffled array, than the procedure is repeated for the remaining array of lenght N-1, until you completely shuffled your array.

2. **Root-finding Methods**
   Implement three of the four sorting algorithms listed below (bonus for all four methods), aimed at

finding a single root in a given interval (or starting point where appropriate). *Hint: you can code your procedure such that one of the parameters passed to it is a generic function. When testing your code (see Points a-d below), you can conveniently define the analytical functions you want to solve through the* `lambda function` *construct available in Python.*

1. Bisection
2. Secant
3. Newton-Raphson
4. False position

To test your methods find the roots of the following functions:

(a) Find the root of a function given by $f(x) = x^3 - 6x^2 + 11x - 6$ for the interval $[2.5, 4.0]$ or near the point 4.0 using the above methods. How many iterations were needed for each of the methods? Which method converged more quickly than the others?

(b) Do the same for a function given by $f(x) = \tan(\pi(x)) - 6$ in $[0.0, 0.48]$ (near 0.48). Which method was the best suited here? Which was the least suited and why? How can you improve the performance of the least efficient method here?

(c) Do the same for a function given by $f(x) = x^3 - 2x + 2$ in $[-2.0, 0.0]$ (near 0.0). Which method was the best suited here? Which was the least suited and why? How can you improve the performance of the least efficient method here?

(d) Do the same for a function given by $f(x) = e^{10(x-1)} - \frac{1}{10}$ in $[0.0, 1.5]$ (near 0.0). Which method was the best suited here? Which was the least suited and why? How can you improve the performance of the least efficient method here?

Solution:

(a) Root is 3.0, Newton Raphson method converges the fastest here.

(b) Root is 0.4474. NR method is fastest and Secant method fails as it diverges away from the root from this limit. Also False position is very slow here.

(c) Root is -1.769. NR method fails as it oscillates between 0 and 1 here. The other methods are also slow for comparatively large difference in initial values.

(d) Root is $x = 0.769741$, this works only for bisection, NR (and Brents).

Advanced exercises

3. **Slice Sampling of a Log-normal Distribution**
In astrophysics the log-normal distribution appears frequently, the PDF of this distribution is given by:

$$f(x) = \frac{1}{x\sigma\sqrt{2\pi}} \exp\left(-\frac{(\ln x - \mu)^2}{2\sigma^2}\right). \tag{1}$$

The mode of the log-normal distribution has value $e^{\mu-\sigma^2}$.

(a) Construct a Xorshift algorithm to construct random numbers.

(b) Using the same random numbers use the bisection method to find the length of the slices and generate random numbers that follow a log-normal distribution and compare them with the log-normal distribution.

(c) Using the same random numbers use the false position method to find the length of the slices and generate random numbers that follow a log-normal distribution and compare them with the log-normal distribution.

(d) Can you think of an other method to construct a log-normal distribution?

4. **Orbital Motion Around a Supermassive Black Hole**
As astronomers we are particularly interested in orbital motions of objects, either in our own solar system, in our own Milky Way or even in other galaxies. Because of this we will discuss an application of root finding which occurs frequently in astrophysics. Let's consider a spherically symmetric potential, in this case it is convenient to use spherical coordinates and therefore write the Hamiltonian as:

$$H = \frac{1}{2}m\left(\dot{r}^2 + r^2\dot{\phi}^2\right) + V(r). \tag{2}$$

We know angular momentum is conserved ($L_z = mr^2\dot{\phi}$), this means that we can write the Hamiltonian as:

$$H = \frac{1}{2}m\dot{r}^2 + \frac{1}{2}\frac{L_z^2}{mr^2} + V(r). \tag{3}$$

It is often convenient to redefine an effective potential as:

$$U(r) = \frac{1}{2}\frac{L_z^2}{mr^2} + V(r). \tag{4}$$

If we know the energy of a planet or star in a potential we can use the Hamiltonian to rewrite the equation to:

$$\dot{r}^2 = 2\frac{E - U(r)}{m} \equiv F(r). \tag{5}$$

So the next question is can we find the point at which the radial velocity is zero, and therefore find a possible orbit. In this exercise use several root finding techniques to show that your result is consistent.[1]

(a) In the case of the Newtonian potential, the potential is given by:

$$V(r) = -\frac{GM}{r}. \tag{6}$$

Assume zero angular momentum. Calculate the radius in the case that we have a star of $2\mathrm{M}_\odot$ around Sagittarius A*. Sagittarius A* has a mass of $4 \cdot 10^6$ $\mathrm{M}_\odot$. Consider that the star has an orbital energy of 10^{24} erg and 10^{20} erg. Calculate the radius in these cases.[2]

(b) Now take into account that the star that circles around Sagittarius A* has angular momentum. Again consider the same orbital energies. For the highest orbital energy consider an angular momentum of $L_z = 2 \cdot 10^{37}$ g cm^2 s^{-1}. For the lower energy consider an angular momentum of $L_z = 10^{38}$ g cm^2 s^{-1}. In the case that the angular momentum is non-zero, there are two roots. Find both roots in both cases and write an algorithm that can find both.

(c) Because we are calculating the orbital parameters of a star around a black hole it is also interesting to take into account the relativistic corrections. In general the relativistic correction term is given by:

$$V_{\text{rel. corr.}}(r) = -\frac{GML_z^2}{c^2r^3}. \tag{7}$$

Calculate the radius in the case that we take into account relativistic corrections for both orbital energies. Consider an angular momentum of 10^{20} g cm^2 s^{-1} for this calculation.

[1] The root finder is up to you, you can use bisection, secant, false position, Brent's method, Newton-Raphson, a combination or even another method.

[2] When you search for roots, always plot your functions!